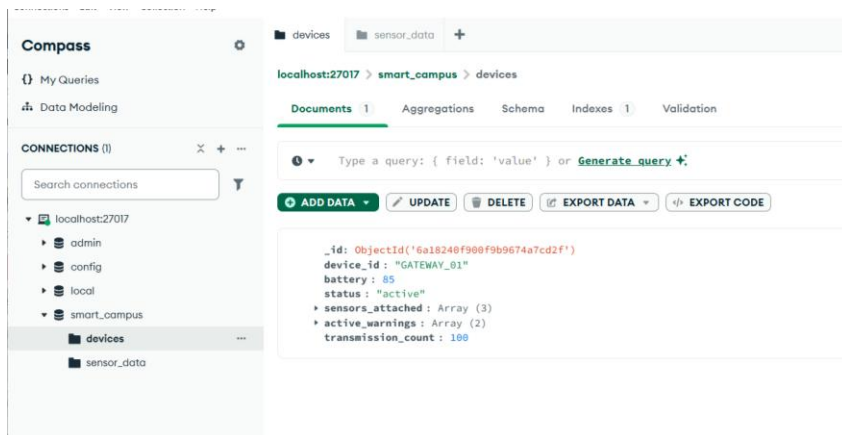
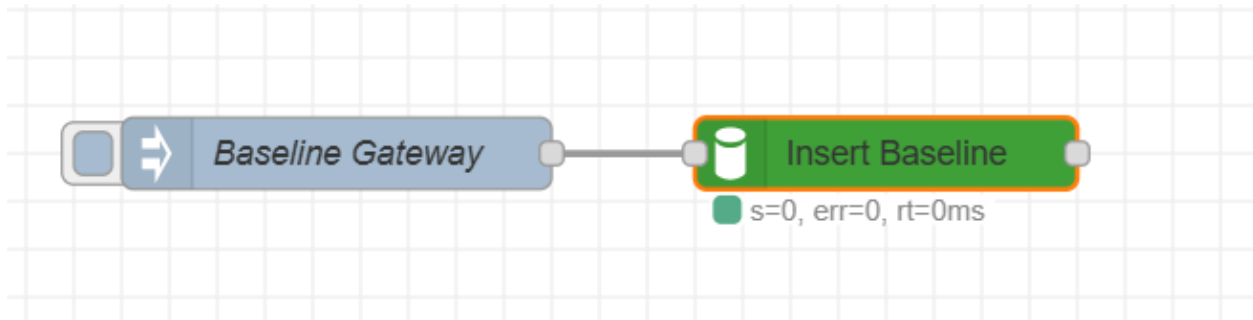
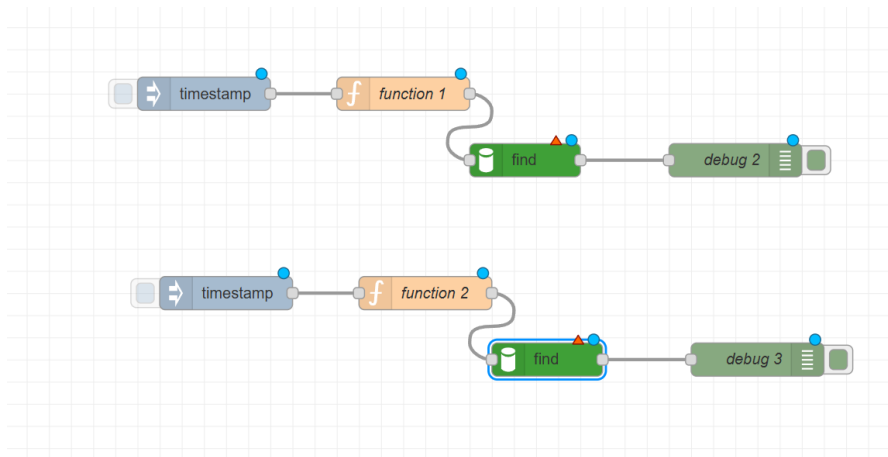


Exercise 7: Advanced NoSQL Query Operators

Step 1: Baseline Data Initialization



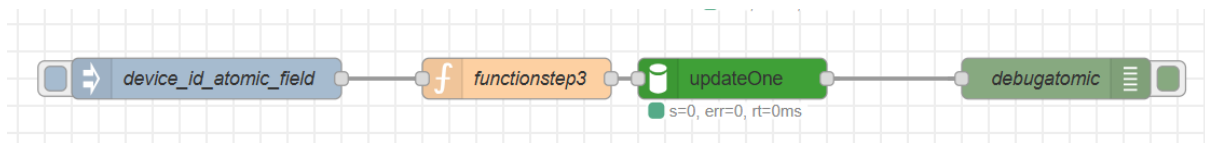
Step 2: Comparison and Logical Operators



```
msg.payload : array[1]
▼ array[1]
▼ 0: object
  _id: "6a18240f900f9b9674a7cd2f"
  device_id: "GATEWAY_01"
  battery: 85
  status: "active"
  ▼ sensors_attached: array[3]
    0: "temperature"
    1: "humidity"
    2: "soil_moisture"
  ▼ active_warnings: array[2]
    ▼ 0: object
      code: 102
      severity: "low"
    ▼ 1: object
      code: 505
      severity: "critical"
  transmission_count: 100
```

```
msg.payload : array[1]
▼ array[1]
▼ 0: object
  _id: "6a18240f900f9b9674a7cd2f"
  device_id: "GATEWAY_01"
  battery: 85
  status: "active"
  ▼ sensors_attached: array[3]
    0: "temperature"
    1: "humidity"
    2: "soil_moisture"
  ▼ active_warnings: array[2]
    ► 0: object
    ► 1: object
  transmission_count: 100
```

Step 3: Atomic Field Modifiers



msg.payload : Object

▼ object

```
acknowledged: true
modifiedCount: 1
upsertedId: null
upsertedCount: 0
matchedCount: 1
```

MongoDB Compass - localhost:27017/smart_campus.devices

Connections Edit View Collection Help

Compass

My Queries

Data Modeling

CONNECTIONS (1)

Search connections

localhost:27017

- admin
- config
- local
- smart_campus
 - devices
 - sensor_data

devices sensor_data +

localhost:27017 > smart_campus > devices

Documents 1 Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#)

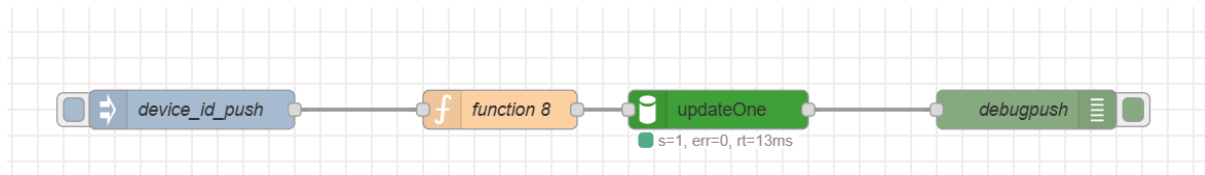
Explain Reset Find Options

ADD DATA UPDATE DELETE EXPORT DATA EXPORT CODE

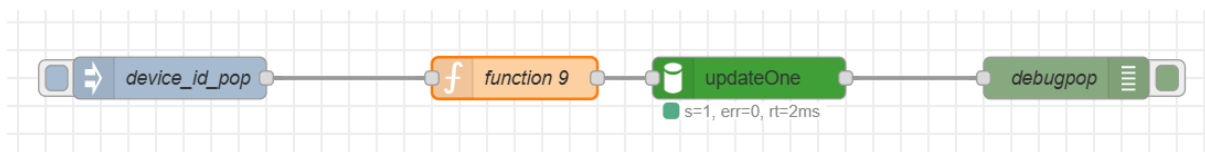
25 1-1 of 1

```
{
  "_id": ObjectId("6a18240f980f9b9674a7cd2f"),
  "device_id": "GATEWAY_01",
  "battery": 82,
  "status": "active",
  "sensors_attached": Array (3)
    0: "temperature"
    1: "humidity"
    2: "soil_moisture"
  "active_warnings": Array (2)
    0: Object
    1: Object
  "transmission_count": 101
}
```

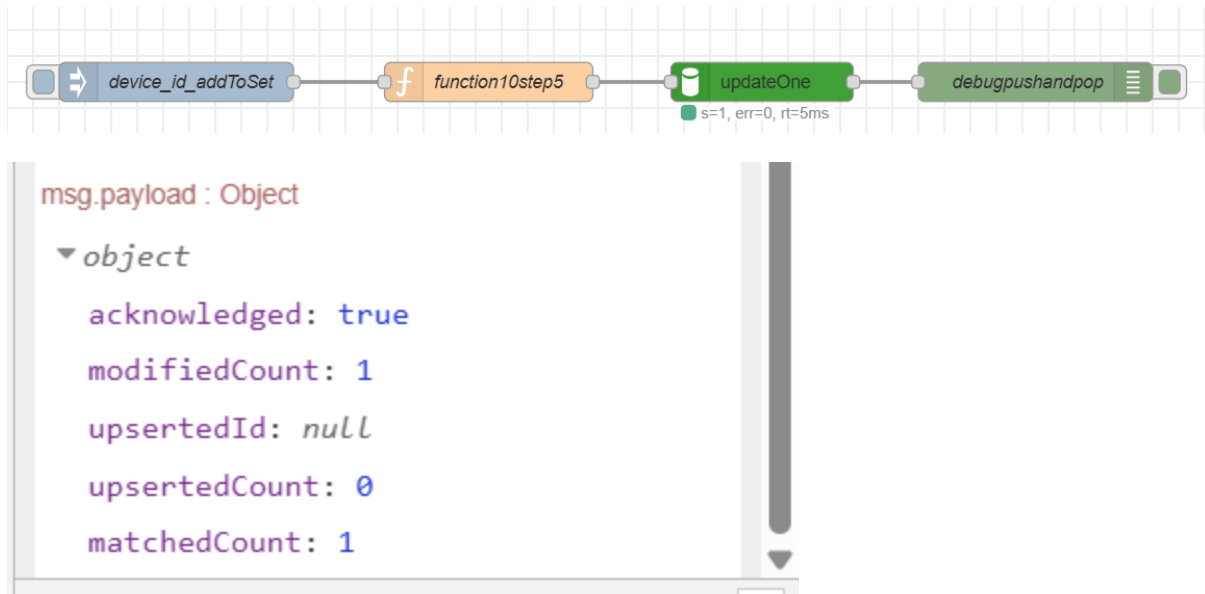
Step 4: Complex Array Manipulation



```
msg.payload : Object
  ▼ object
    acknowledged: true
    modifiedCount: 1
    upsertedId: null
    upsertedCount: 0
    matchedCount: 1
```



```
msg.payload : Object
  ▼ object
    acknowledged: true
    modifiedCount: 1
    upsertedId: null
    upsertedCount: 0
    matchedCount: 1
```



Part 5: Discussion & Architectural Analysis

Question 1: The \$inc vs. \$set Paradigm

The \$inc vs. \$set Paradigm: Using native atomic operations avoids **lost update anomalies** (race conditions) caused by the latency between reading data into Node-RED and writing it back. Native modifiers handle calculations inside a single transaction pass at the storage level.

Question 2: Efficient Payload Management

Efficient Payload Management: Utilizing native array modifiers (like \$push or \$addToSet) significantly optimizes network performance. Instead of downloading an entire array over the network, mutating it in Node-RED's memory, and re-uploading it, Node-RED only sends a tiny instructional delta string. The database engine does the heavy lifting locally.